Campus Information Retrieval Assistant (CIRA)

A Major Project report submitted
in partial fulfillment of requirement for the award of degree

BACHELOR OF TECHNOLOGY
in
SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE
by

CH. JIGNESH SHOURYA	**(2303A51L90)**
D. MARUTHI	**(2303A51L98)**
K. SINDHU	**(2303A51LA0)**
B. JAYANTH	**(2303A51LA7)**
G. MAHENDRA	**(2303A51LA9)**
K. NIKHIL	**(2303A51LB0)**

Under the guidance of
Mr. Gotte Ranjit Kumar
Assistant Professor, School of Computer Science and Artificial Intelligence.

SR UNIVERSITY

SR University, Ananthasagar, Warangal, Telangana - 506371

SR University

Ananthasagar, Warangal.

SR UNIVERSITY

CERTIFICATE

This is to certify that this project entitled **"Campus Information Retrieval Assistant (CIRA)"** is the Bonafide work carried out by **CH. Jignesh Shourya, D. Maruthi, K. Sindhu, B. Jayanth, G. Mahendra, K. Nikhil** as a Major Project for the partial fulfillment to award the degree **BACHELOR OF TECHNOLOGY** in **School of Computer Science and Artificial Intelligence** during the academic year 2025-2026 under our guidance and Supervision.

Mr. Gotte Ranjit Kumar
Assistant Professor,
SR University
Ananthasagar, Warangal.

Mr. Sandeep Chintham
Professor & Head,
School of CS&AI,
SR University
Ananthasagar, Warangal.

Reviewer-1
Name:
Designation:
Signature:

Reviewer-2
Name:
Designation:
Signature:

ACKNOWLEDGEMENT

We owe an enormous debt of gratitude to our Major project guide **Mr. Gotte Ranjith Kumar, Assistant Professor** as well as Head of the School of Computer Science and Artificial Intelligence, **Mr. Sandeep Chintham, Professor** for guiding us from the beginning through the Major Project with their intellectual advice and insightful suggestions. We truly value their consistent feedback on our progress, which was always constructive and encouraging and ultimately guided us in the right direction.

We express our thanks to project coordinator **Mr. Sallauddin Md, Assistant Professor** for their encouragement and support.

Finally, we express our thanks to all the teaching and non-teaching staff of the department for their suggestions and timely support.

TABLE OF CONTENTS

S.No.	Content	Page No.
	Abstract	1
I	Introduction	2
II	Related Work	3
III	Problem Statement	5
IV	Requirement Analysis	7
V	Proposed System	10
VI	Architecture Diagrams, Flow Charts, DFD	13
VII	Simulation Setup and Implementation	27
VIII	Result Comparison and Analysis	42
IX	Learning Outcomes	45
X	Conclusion with Challenges	46
XI	References	48
XII	Project Artifacts	51

LIST OF FIGURES

S.No.	Figure Name	Page No.
1	Architecture Diagram	13
2	Flowchart	15
3	Data Flow Diagram	17
4	Class Diagram	19
5	Activity Diagram	21
6	Overall System Workflow	23
7	CIRA User Interface / Web Landing Page	38
8	Chat Interface: Hierarchy Search, Faculty Search Information Search & Place Search	44

LIST OF ACRONYMS

Abbreviation	Full Form
API	Application Programming Interface
BM25	Best Matching 25 (Ranking Algorithm)
CIRA	Campus Information Retrieval Assistant
CS&AI	Computer Science and Artificial Intelligence
CSE	Computer Science and Engineering
DFD	Data Flow Diagram
EEE	Electrical and Electronics Engineering
FAISS	Facebook AI Similarity Search
FAQ	Frequently Asked Questions
GPT	Generative Pre-trained Transformer
HOD	Head of Department
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
PDF	Portable Document Format
QA / Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
SLM	Small Language Model
SRU	SR University
UI	User Interface
UX	User Experience
DB	Database
ML	Machine Learning
IR	Information Retrieval
NLU	Natural Language Understanding
CPU	Central Processing Unit
GPU	Graphics Processing Unit

ABSTRACT

Universities have accumulated a body of information from various sites, research papers, announcements, handbooks, and FAQs. As such, students and other users cannot access this information quickly. As such, users are forced to spend a lot of time searching through various sites to access basic information about the university. However, other problems have been identified. These include increased costs, privacy issues, and dissemination of unverified information that a conventional cloud-based chatbot with large models can disseminate. To solve these problems, we propose a solution called the Campus Information Retrieval Assistant (CIRA). This is a smart assistant that can access accurate and reliable information about the university. This is achieved through accessing information from reliable sources and then generating responses from this information only. These assistant accesses information from various documents available on the university's campus. For instance, information from PDF files is accessed first. FAQs and other sources are used only as a last resort. The assistant can improve user privacy using university infrastructure. As such, users can interact with the assistant through a natural language interface. This allows users such as students and visitors to interact with the assistant and access information about facilities and other information available on campus. This assistant can offer users fast responses to any query. This is achieved with lightweight language models and knowledge grounding. As such, this assistant can offer rapid responses to users and improve user experience.

I. INTRODUCTION:

Universities are complex information environments. Universities produce a tremendous volume of information on academics, administration, infrastructure, and student services. This information is scattered on a variety of websites, policy documents, student guides, circulars, and departmental notifications. Although all these sources are rich in information, the way information is scattered makes it difficult for students, faculty, and visitors to find information quickly.

The requirement for rapid and reliable information access is on the increase in the context of the increasing digitalization in the field of higher education. Conventional on-site search facilities on university websites mostly use keyword inputs and a predefined navigation system. These facilities may help find documents, but they may not always provide concise and direct information. This is particularly difficult for new students and visitors who are not familiar with the way information is organized.

Recent developments in intelligent information retrieval systems are promising to help achieve this in a much easier manner through interaction with such systems using natural language. Instead of searching through documents to obtain information, you simply ask your query and obtain an accurate answer from relevant institutional documents. This means that such systems are associated with increased costs and associated privacy and control issues when such information is required to be processed on the cloud.

To address these challenges associated with such systems, we propose a solution called the Campus Information Retrieval Assistant (CIRA), which is a campus-based information access system intended to help users obtain accurate information through verified institutional information. The system is based on a Small Language Model (SLM) and a retrieval-based mechanism to obtain relevant information from trusted institutional documents. The primary objective is to help users access information on the campus in a much easier manner through an accurate and user-friendly interface. By providing users with the ability to interact with such systems using natural language and institutional documentation as a primary source of information, such systems aim to reduce information search time and improve user experience.

II. RELATED WORK

In recent times, there is a need for information accessibility in schools and universities, hence the need for conversational information systems that can assist students and administrators. Recently, researchers have been combining document information systems with language processing techniques for better accuracy and reliability of information.

Nguyen-Duc et al. (2025) performed an empirical study that focused on the application of multi-agent information retrieval systems for university admissions counseling services. The authors of the study found that the application of multiple information retrieval agents for information extraction from documents can enhance the accuracy of information. The authors found that the application of information retrieval systems for academic counseling services makes the system more reliable and reduces the occurrence of misleading information.

Oh and Kim (2025) proposed a system for answering questions for university academic administration services. The proposed system is based on information retrieval systems that extract information from relevant documents for the purpose of answering questions by students and administrators. The authors of the study found that the application of information retrieval systems for academic administration services is precise and accurate.

Xu et al. (2025) proposed a framework called MEGA-RAG that enhances the reliability of information retrieval systems through the application of multiple evidence sources during information generation. The proposed system is based on information retrieval systems that enhance the reliability of information through a multi-step verification process. The proposed system is based on public health information systems, but it is clear that the application of evidence-based information retrieval systems can reduce the occurrence of misleading information.

Hartono, Setiawan, and Singh (2025) developed a chatbot that can assist prospective and current students in universities. It utilized data from universities, including admission policies, courses, and information about universities. This research shows that retrieval-based chatbots can assist in accessing information about universities and can accelerate information search.

Swacha and Gracel (2025) provided an extensive survey on retrieval-based chatbots in an educational environment. They emphasize how retrieval-based chatbots can assist in improving digital support in an educational environment. This research shows that retrieval-based chatbots can improve digital support in an educational environment.

However, apart from retrieval-based systems, recent research has also explored lightweight language processing models that are easy to implement. For instance, in a survey of compact language models with good performance using significantly less computational resources than larger models, Subramanian, Elango, and Gungor (2025) found that these models can be implemented in various applications where computational resources are scarce.

In another study, Lamaakal et al. (2025) explored the potential of using tiny language models in automaton and control applications. In the research, the authors discussed how these models can be used in intelligent systems of decision-making in applications with quick response times and deployment in a particular environment. These models are therefore suitable for use in institutional infrastructures.

In a recent study, Zhang et al. (2024) presented an open-source compact language model called TinyLlama. It was created with the objective of using language processing efficiently without compromising performance in language tasks. This research indicates that these models can be used in applications with scarce computational resources.

Although various studies have explored retrieval-based conversational systems and lightweight language models separately, none have explored the integration of both in a campus-specific context of information retrieval. The Campus Information Retrieval Assistant combines a lightweight language processing model with a retrieval-based model with a focus on institutional documents. It therefore provides accurate and efficient results in a manner that respects user privacy.

III. PROBLEM STATEMENT

The problem is that modern universities manage a large volumes of information, and it's scattered all over the place. There's information for students, information for administrators, information for the university, and information for the facilities and services provided by the university. All of it is scattered all over the place, and while it's all intended for the benefit of students, it is challenging to get the information you need.

The problem is exacerbated by large universities that have many departments and administration groups that each manage their own information. So, while each department and administration group may be providing useful information, it's scattered all over the place, and it is challenging to get the information you need.

The traditional search systems that universities provide are usually based on a keyword search system, and while that's good for getting information from documents that contain something related to your query, it is challenging to get direct answers from them. In fact, it is challenging to get any answers from them, and that's a problem, as it might drive people away from using the information that's provided by the university.

This is where conversational assistance systems may be able to help us improve information access. The problem is that many existing conversational systems are built on top of large computing workloads running in the cloud. This has its own set of problems, such as high operating costs and possible privacy breaches. For educational institutions, there is also the added problem of needing to process information within an institution's network.

Another problem is that conversational systems are only as good as the information provided to them. Since this information is not necessarily tied to official institutional sources, there is a risk that the information provided to users may be inconsistent with official documents or policies. This is especially true if the system is used to answer questions that are not within its originally established knowledge domain. For an educational institution, such inconsistencies may be particularly problematic because accuracy is crucial to user acceptance and institutional communication.

Finally, existing systems are generally not tailored to a university's information structure. As such, these systems cannot meet our unique information needs as an educational institution.

There is a genuine need for a campus-centric information retrieval system that is both efficient and reliable. The notion is that users can ask a question in a natural way and receive answers that are grounded in official sources of information. However, it is also important that it can function smoothly within a university setting while ensuring data privacy and reducing dependence on external services.

To solve this problem, it is important that a proper approach is adopted that incorporates smart information retrieval along with effective language processing. In this way, it is possible to enrich the university experience for students by improving access to campus information.

IV. REQUIREMENT ANALYSIS

The system is intended to give quick answers to users' natural language queries related to the campus, its facilities, academic departments, faculty, policies, administrative procedures, etc. Instead of searching a heap of documents or surfing the internet for answers, users can pose a query in plain language and get a clear, concise answer. In short, the system fetches information from reliable sources, such as PDFs, FAQs, and official documents, and provides a clear, concise answer to the user's query through the interactive interface.

4.1. PERFORMANCE REQUIREMENTS

1. **NLP SUPPORT**
    - Users can query via natural language.
    - It can answer queries about departments, faculty, facilities, policies, and procedures.
    - It will analyze queries and determine where relevant campus information is.
2. **THE RETRIEVAL SYSTEM**
    - The system, called the Campus Information Retrieval Assistant, is intended to fetch information from reliable sources, such as PDF documents, FAQs.
    - The official documents from the campus, and give a clear, concise answer to the user.
3. **DATA ORGANIZATION**
    - Campus information will be organized in a repository. -
    - Documents will be indexed for fast retrieval.
    - It will strive for fast access in query processing.
4. **USER INTERFACE**
    - The system provides an interface that’s clean and interactive.
    - This makes it very easy to interact with the system regardless of your technical knowledge.
    - You can easily submit your query.

5. **OUTPUTS**
    - The outputs are clearly presented and unambiguous.
    - The outputs are related to various aspects of the campus like locations, faculty, departments, etc.
    - The outputs are derived from reliable documents.

4.2. NON-PERFORMANCE REQUIREMENTS

1. **EXECUTION**
    - Quick response will be given for queries with minimal delay.
    - It will efficiently handle queries without causing delay.
2. **SERVICEABLENESS**
    - It's simple and easy to use.
    - There's no need to have any technical knowledge whatsoever.
3. **RELIABILITY**
    - The system retrieves information related to the campus from reliable sources.
    - The system offers accurate information related to the campus by using reliable documents.
4. **SAFETY**
    - The system handles the information related to the campus safely.
    - Safety is the topmost priority for the system while handling the information.
5. **EXTENSIBILITY, OPERABILITY AND MAINTAINABILITY**
    - The system is designed to be extended to include any updates or new documents on the campus.
    - The system can be extended to include new sources of information swithout any effect on the existing system.

RISK ANALYSIS

The Campus Information Retrieval Assistant has a few risks as it is developed and implemented. Firstly, the data that you are relying on is important. This is because the data that the system will be based on might be incomplete or obsolete. The way to avoid this is to ensure that the system is based on official Institution Documents.

FEASIBILITY ANALYSIS

This section checks if the proposed system is viable given the resources that are available.

1. TECHNICAL VIABILITY

- From a technical point of view, it is viable since we are using technology that is already proven for processing documents, information retrieval, and querying of natural language.
- The system can be implemented using existing technology that is already present on the campus, requiring no major computing resources.

2. PRACTICAL USABILITY

- The system is designed to be extremely practical and easy to use.
- The system is accessible for students, staff, and visitors on campus through natural language queries, requiring little or no training.

3. COST EFFECTIVENESS

- The system is cost-effective since we do not need any external services or cloud computing resources that are costly.
- The cost of maintaining this system is minimal, requiring occasional updates of documents that are present on the campus.

4. LEGAL VALIDITY

- The system is legal since everything we are doing is within the legal framework of the institution.
- The information that we are using is verified information, and proper handling of data ensures our security.

5. CULTURAL VIABILITY

- The system is viable since our primary objective is to ensure that information is accessible to everybody on campus.
- The system is designed in a way that information is readily available, creating a smooth information environment that is very user-friendly.

V. PROPOSED SYSTEM

The Campus Information Retrieval Assistant is a tool designed to allow the user to access information from the university quickly and efficiently. It does this by using one interface that retrieves information from the university. Rather than relying on one source of information, the system uses a combination of data sets, which include information regarding the university's faculty, the location of the university, frequently asked questions, and official documents. All this information is used to retrieve accurate information quickly.

1. DATA COLLECTION AND ORGANISATION

The first part of the process involves the collection of data from the university's site. Pages that contain information regarding the university's faculty and the university's hierarchy are collected and then organized. This includes information regarding the university's faculty, their names, their positions, and the departments they head. Organizing the data allows the system to answer questions regarding the university's faculty without processing the data.

2. CAMPUS LOCATION INFORMATION

- To improve navigation on campus, key locations are identified and stored.
- These locations are stored in a structured format as JSON objects.
- This dataset allows the assistant to process navigation-related questions.

3. FAQs

- A set of questions related to common queries is created. This allows the assistant to provide answers to common questions asked by users.
- These questions are usually related to common topics that are asked by users.
- This module allows the assistant to quickly respond to common questions by matching the questions with the stored FAQ data.

4. DOCUMENT KNOWLEDGE BASE

- The knowledge base for this module includes a set of official documents available on the university's site. Official documents include guidelines, notifications, and other information available on the site.
- These documents are usually available as PDF files. These files are stored as text.
- The content retrieved from the documents is combined into a single text corpus. This corpus acts as a knowledge source for answering questions.

5. SEMANTIC RETRIEVAL WITH VECTOR SEARCH

For better information retrieval, the text information is indexed for vector search. We use semantic embeddings for the information in the text, and these embeddings are stored in a FAISS index.

This allows us to conduct vector searches when users ask complicated questions. Even when the user's question is not exactly similar, we can still identify the relevant part of the information in the text.

6. RESPONSE GENERATION USING A COMPACT LANGUAGE MODEL

The final response is generated by a SLM. This response is based solely on the retrieved information. Since everything is based on information stored locally, the information provided is accurate and consistent with the institution.

7. SYSTEM FLOW:

i) **Input**
The user asks the assistant for information related to the campus.

ii) **Data retrieval**
The system analyzes the user's query and searches for information in various sources, such as faculty information, location information, FAQs, and document information.

iii) **Semantic search**
If information from documents is required, the system uses the FAISS vector search to retrieve information from the knowledge base.

iv) **Response generation**
The information retrieved is fed into the language model, and a meaningful response is generated.

v) **Results**
The final response is provided to the user by the assistant.

8. ARCHITECTURE OF SYSTEM

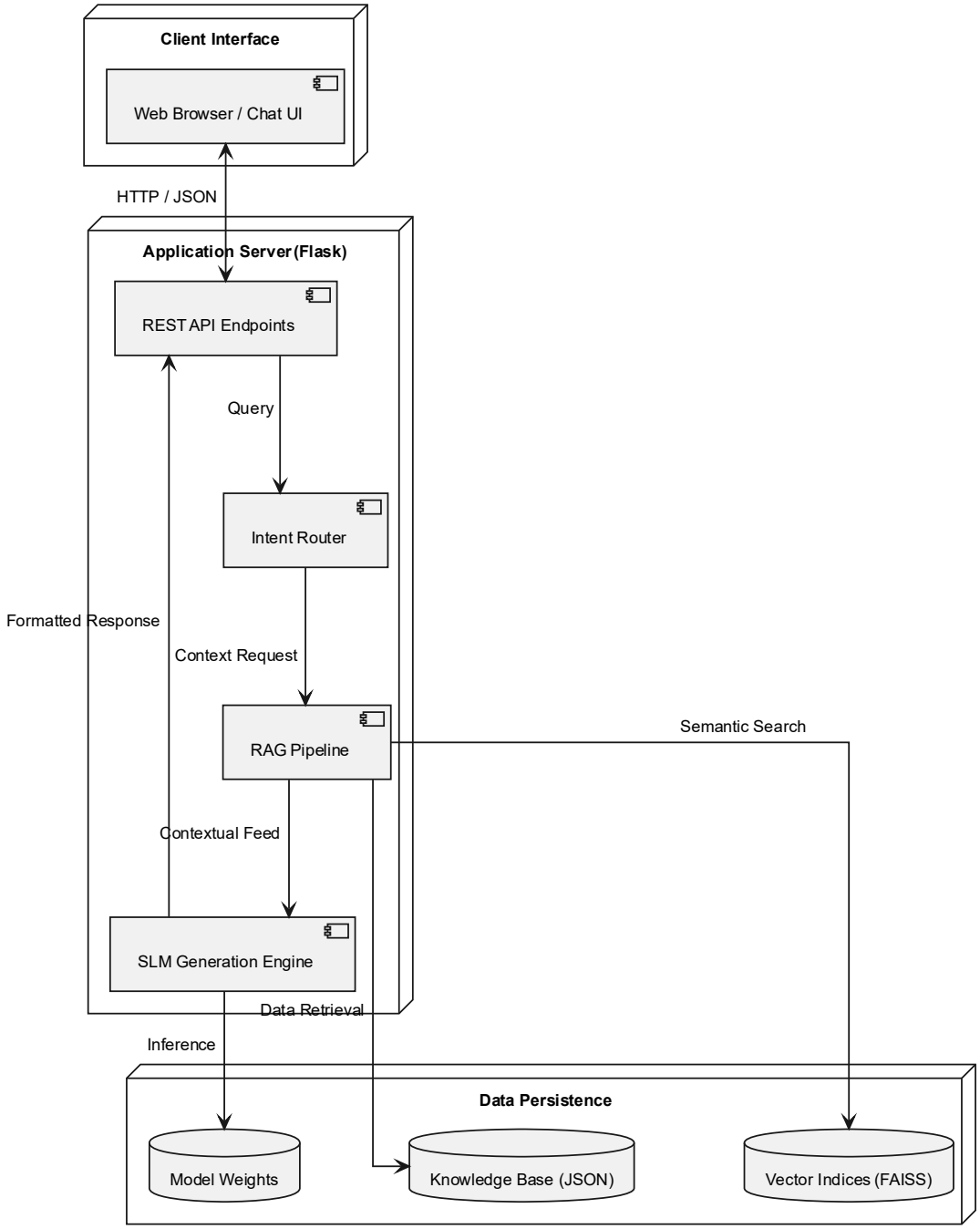
- The system is composed of various parts that are specialized in what they do.
- It includes structured data sets for faculty and hierarchy, location data to enable you to wander the campus, structured data sets for your most frequently asked questions, and a set of documents for more extensive information retrieval.
- The modules are independent, which means that if one module changes, the others will not be affected. This improves the efficiency, scalability, and reliability of the system.

9. SECURITY

- We use validated data sets from various university sources to ensure the accuracy of the data you receive.
- Since the assistant works from local data sets, we have control over the use of the data and do not rely on unverified external sources.

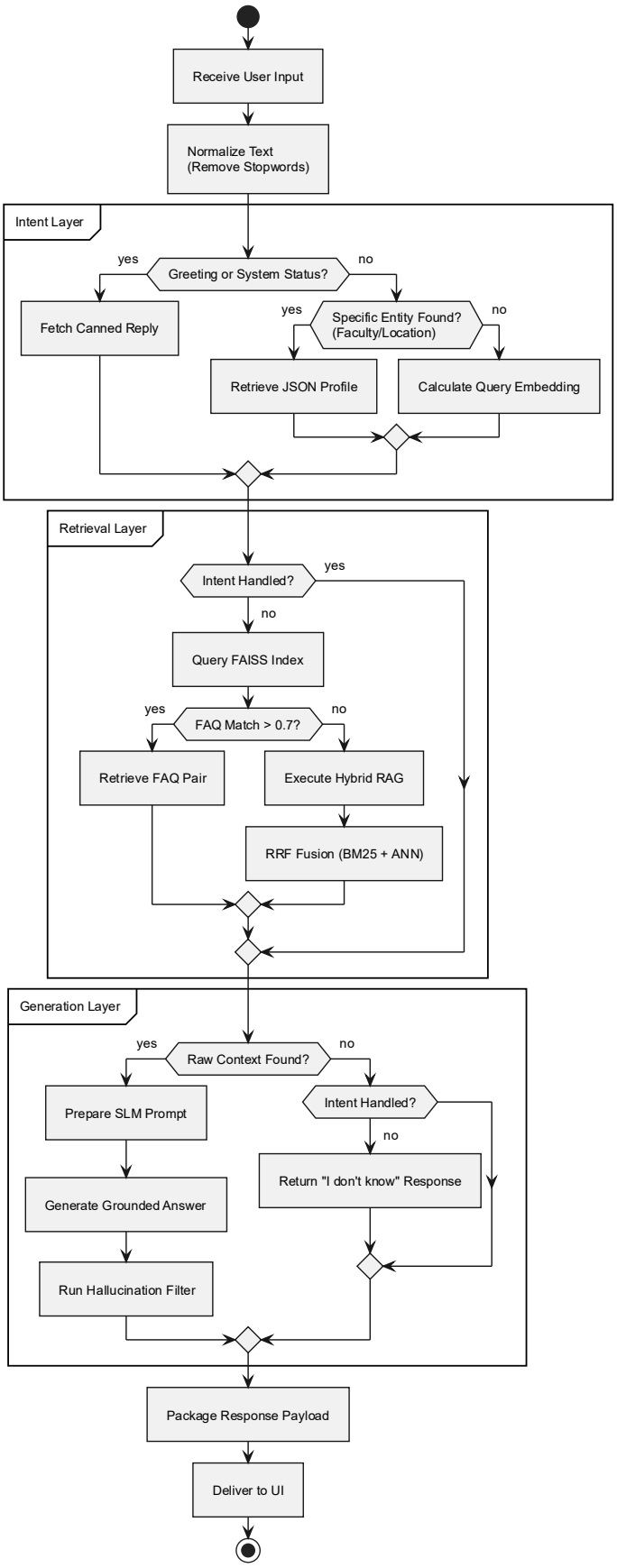
VI. SYSTEM DESIGN USING UML DIAGRAMS

ARCHITECTURE DIAGRAM:



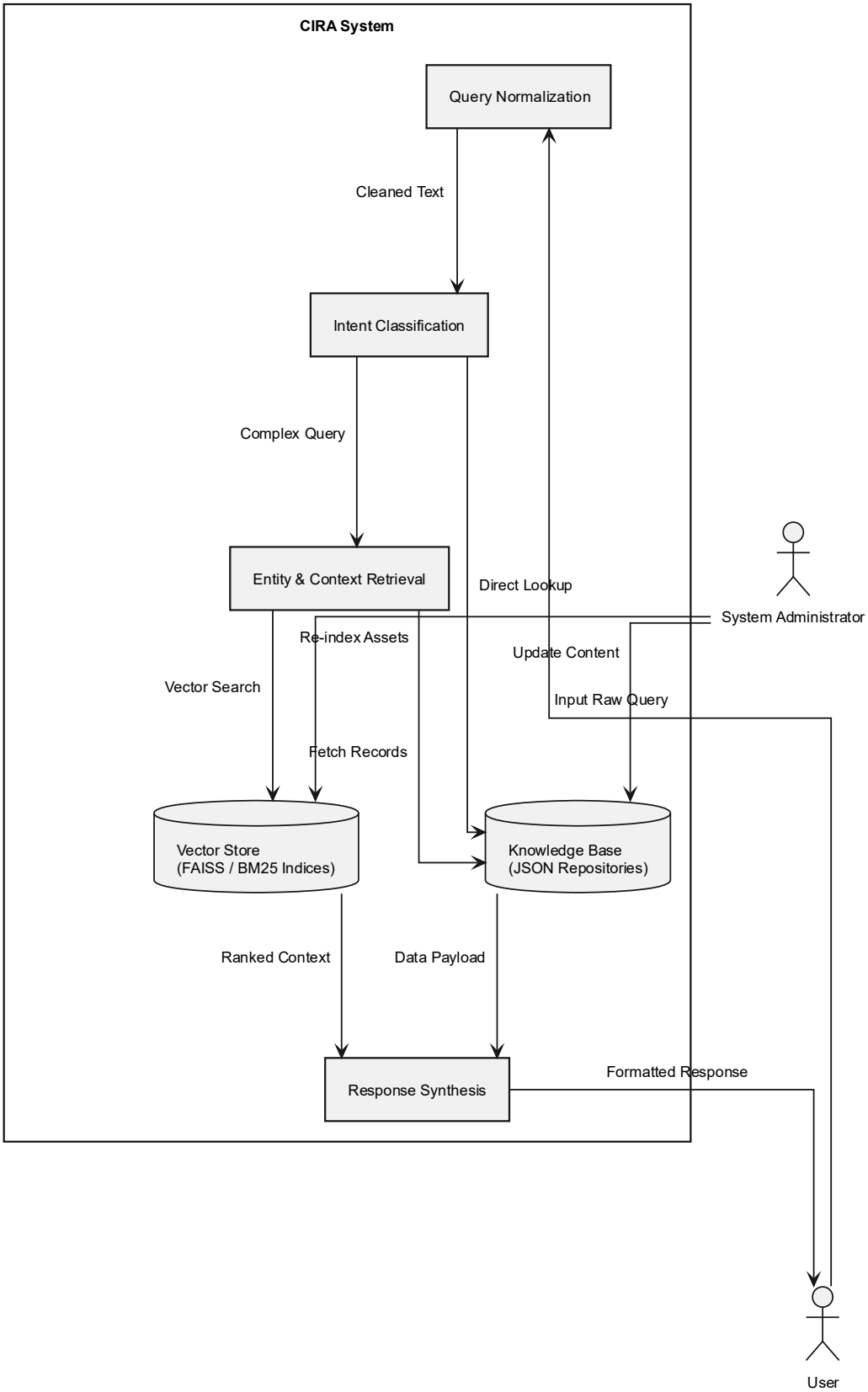
1. **User Interface Layer**
    - It acts as the gateway for the system.
    - Users input their questions in the chat, and the system directs them to the backend.
2. **Query Processing Layer**
    - The backend receives the query and initiates preprocessing.
    - After that, the system figures out what the user wants.
3. **Information Routing Module**
    - Depending on the user intent, the system directs the query to the information source.
    - This could be structured datasets, FAQ, or document retrieval.
4. **Retrieval Layer**
    - It searches the structured datasets for information about the faculty, organizational structure, and location.
    - If the query is too complicated, the system performs semantic retrieval using the FAISS vector index.
5. **Response Generation Module**
    - It structures the information into a response in the form of a string.
    - A lightweight language model is used to organize the information.
6. **Data Storage Layer**
    - It stores various datasets such as faculty information, location, FAQ, and document corpus.
    - It stores the semantic embeddings of the document corpus in the FAISS index.

FLOW CHART


Receive User Input
Normalize Text
(Remove Stopwords)
Intent Layer
Greeting or System Status?
yes
no
Fetch Canned Reply
Specific Entity Found?
(Faculty/Location)
yes
no
Retrieve JSON Profile
Calculate Query Embedding
Retrieval Layer
Intent Handled?
yes
no
Query FAISS Index
FAQ Match > 0.7?
yes
no
Retrieve FAQ Pair
Execute Hybrid RAG
RRF Fusion (BM25 + ANN)
Generation Layer
Raw Context Found?
yes
no
Prepare SLM Prompt
Intent Handled?
no
Return "I don't know" Response
Generate Grounded Answer
Run Hallucination Filter
Package Response Payload
Deliver to UI

1. **Start of Process:**
    - The process starts when the query is submitted in the User Interface by the User.
2. **Preparing the Input:**
    - The query will go through a process to clean and prepare it for analysis.
3. **Greeting Check:**
    - A response will be provided if the query is a greeting.
4. **Faculty Information Check:**
    - If the query pertains to faculty information, the system will retrieve information from the faculty metadata dataset.
5. **Location / Campus Navigation Check:**
    - When the query pertains to campus navigation, the system will retrieve the location coordinates and display them on a map.
6. **FAQ Matching:**
    - The query will be checked to match existing FAQ records.
7. **Semantic Search:**
    - When the query does not match any existing records, a semantic query will be performed using the FAISS vector index.
8. **Crafting a Response:**
    - A reply will be created based on the retrieved document segments.
9. **Presenting the Output:**
    - The final query result will be presented to the user.
10. **End of Process:**
    - The process will end and await the next query.

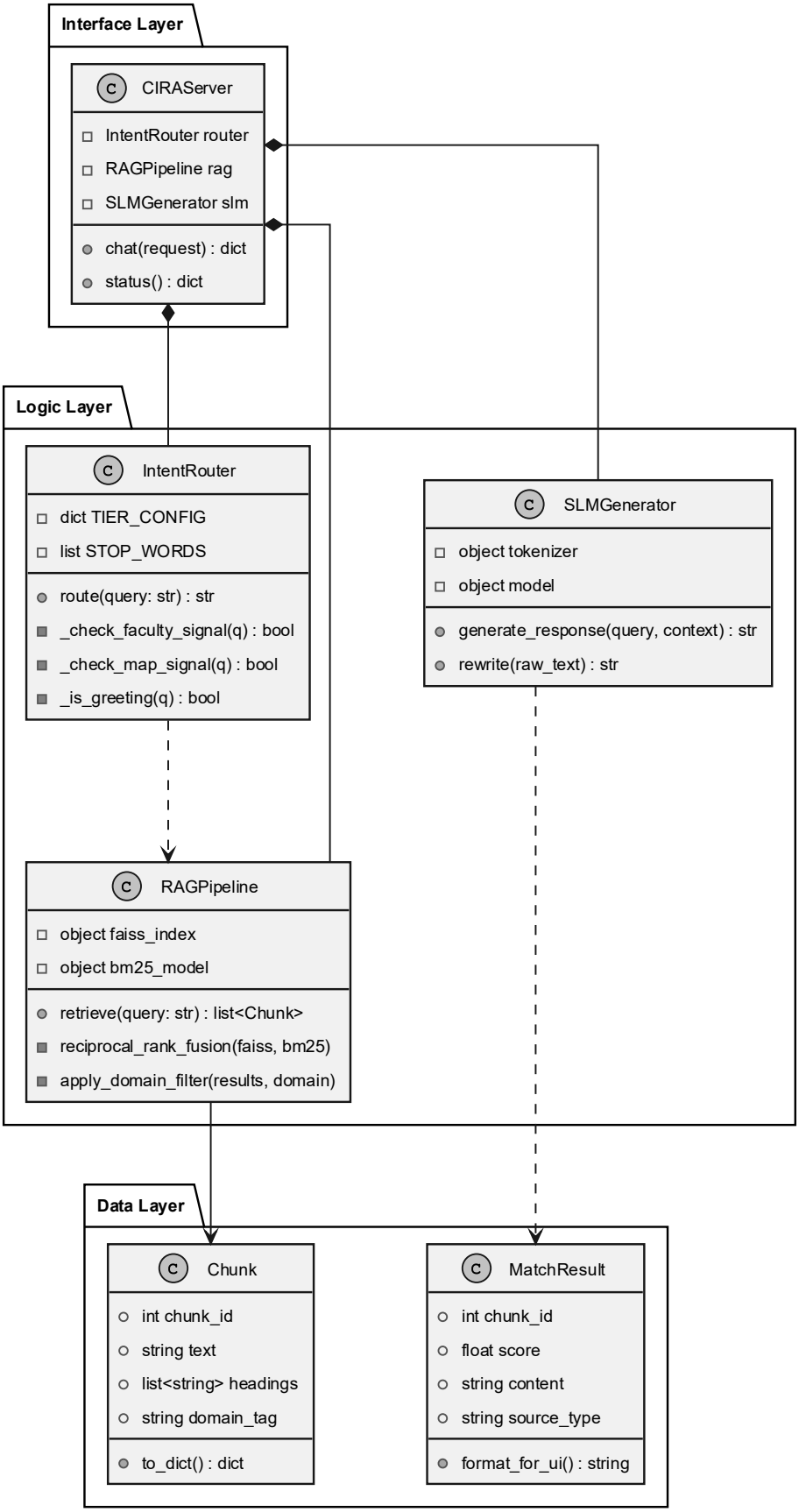
DATAFLOW DIAGRAM:



Process Flow Explanation:

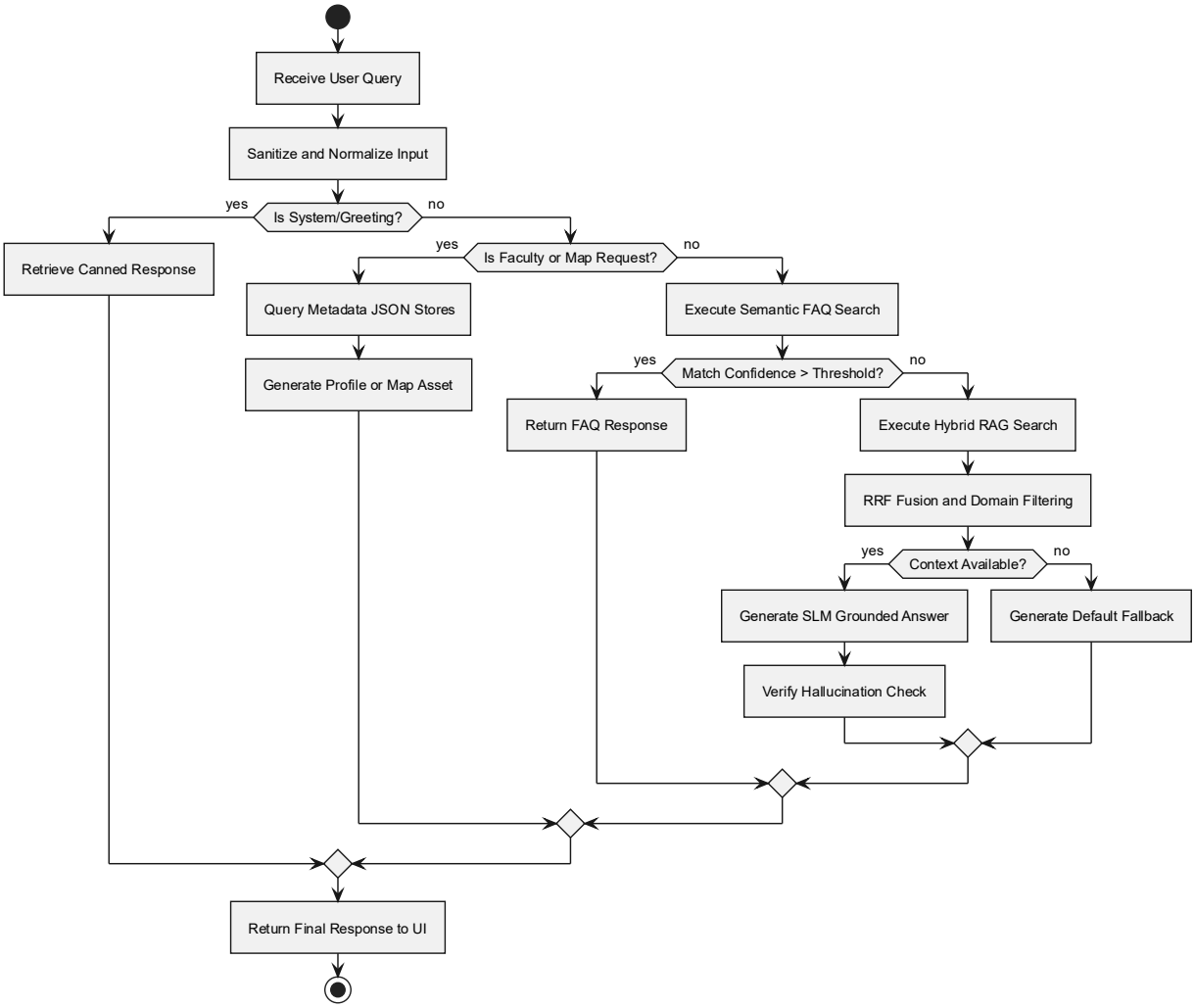
1. **Query Entry:**
    - The user will input a search query through the web interface.
    - This will be the main input to the system.
2. **Query Handling:**
    - The system will execute its preprocessing steps on the input query.
3. **Intent Detection:**
    - The system will identify the type of query category and information required.
4. **Source Selection:**
    - The type of query will determine the source of information to be used.
    - The source will include faculty metadata, location information, FAQ databases, and documents.
5. **Semantic Search:**
    - For complex queries, a transformation will be done to convert the query to vector embeddings.
    - The FAISS index will then fetch relevant sections of documents that are semantically similar to the query.
6. **Information Synthesis:**
    - The information obtained from all sources will be integrated to form a response.
7. **Response Delivery:**
    - The final results will be sent to the UI, then the user can view the displayed results.

CLASS DIAGRAM:



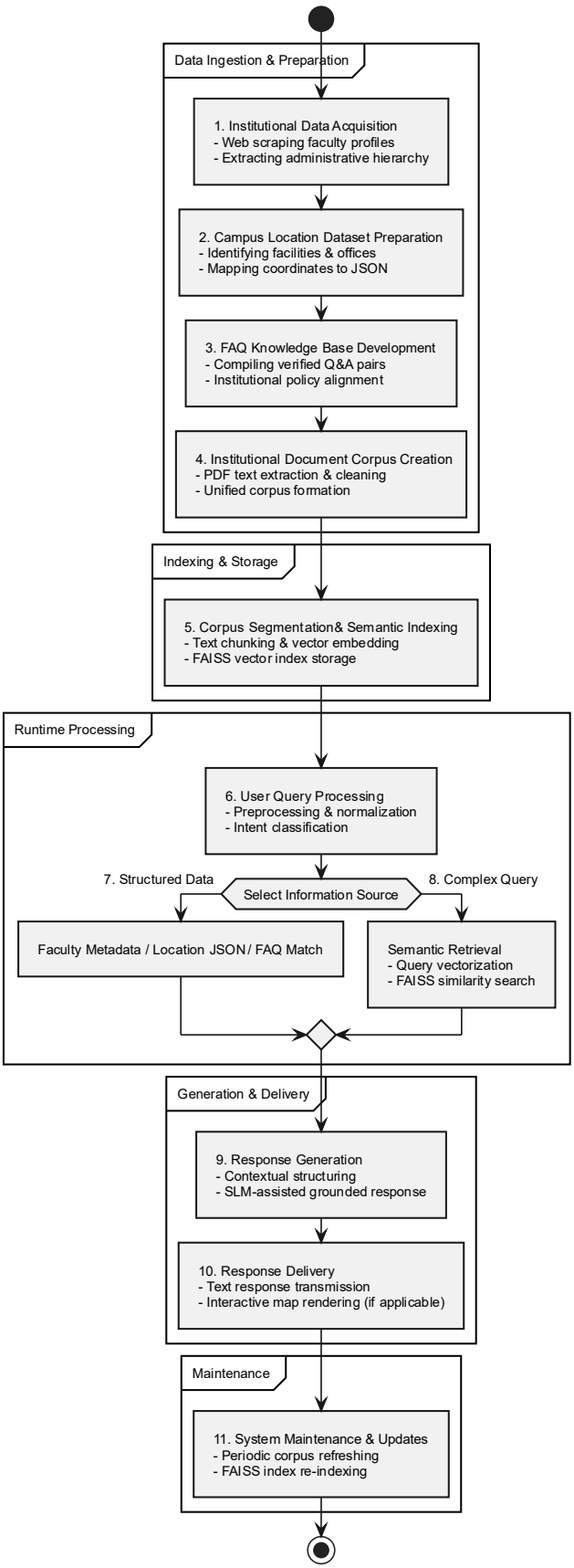
1. **CIRAServer Class:**
    - This class acts as the controller for the entire system, as it is responsible for every operation.
    - It receives user queries and manages the interaction with other classes in the system.
2. **IntentRouter Class:**
    - This class will analyze all queries received from users to identify their actual intentions behind asking those queries. -
    - This class will route all queries to the corresponding processing classes in the system.
3. **RAGPipeline Class:**
    - This class will fetch relevant information from the institutional corpus using semantic search techniques.
4. **SLMGenerator Class:**
    - This class will generate a response to queries in a natural language after obtaining relevant context from the RAGPipeline class.
5. **Data Classes:**
    - **FacultyMetadata:** This class will store information about all faculty members in the system.
    - **LocationData:** This class will store geographical coordinates of all locations in the campus area.
    - **FAQEntry:** This class will store all queries and answers from users in the system.
6. **Relationship Between Classes:**
    - The CIRAServer class will utilize the IntentRouter class to classify queries received from users.
    - The IntentRouter class will utilize the RAGPipeline class to fetch relevant information from the system.
    - The RAGPipeline class will provide the relevant information to the SLM Generator for successful response generation.

ACTIVITY DIAGRAM:



1. **Query Input:**
   - A user enters a query into the system.
2. **Input Preprocessing:**
   - The system cleans the input, removing any “noise” from the input.
3. **Query Classification:**
   - The system goes through the query and classifies the type of query it is: greeting, info about the faculty, location, or general info.
4. **Direct Information Retrieval:**
   - If the query is asking about the faculty or leadership, the system retrieves the information from the metadata.
5. **Location Retrieval:**
   - If the user is asking for directions, the system retrieves the coordinates from the location dataset.
6. **FAQ Matching:**
   - The query is compared to a pre-defined list of FAQs.
7. **Document Retrieval:**
   - If the query does not match any of the above, the system uses semantic search on the document dataset.
8. **Response Preparation:**
   - The information that has been retrieved is formatted into a cohesive answer.
9. **Response Delivery:**
   - The answer is then sent to the user interface by passing the queries and displays the answer.

OVERALL SYSTEM WORKFLOW OF CIRA:



The Campus Information Retrieval Assistant follows a series of interconnected steps from data collection to answering user queries. The system combines structured data, document-based knowledge, and a light language model to provide accurate information from campus sources.

1. **Collecting Institutional Data:**
   - The system begins by collecting information from the university's website.
   - The clean structured data helps the system to directly answer user queries about professors and administrators.
   - First, we acquire data from official sources within the university.
   - Information on the faculty and the organizational structure of the university itself is obtained by scraping the official site.
   - The data obtained is then structured to include fields such as name, designation, department, school, and profile link.
   - This allows for an easier way to retrieve answers to questions on the faculty and the organizational structure.
2. **Preparing the Campus Location Dataset:**
   - The system identifies important campus locations such as academic blocks, libraries, hostels, and administrative offices.
   - Each campus location is stored in a structured JSON data type with attributes such as place name and coordinates.
   - The system can now directly answer user queries about campus locations by displaying them on an interactive map.
3. **Building the FAQ Knowledge Base:**
   - The system collects common user queries about academic policies, grades, admissions, and campus facilities.
   - The system then collects verified answers to these queries from relevant sources.
   - The FAQ dataset helps the system to directly answer user queries using exact matches from this dataset.
4. **Creating the Institutional Document Corpus:**
   - The system collects a number of PDF files from the university website to form the depth of its knowledge base.
   - The system then collects text from these documents after removing unwanted information from the documents.

- The clean text from all documents is then merged to form a corpus that contains knowledge from all documents.

5. **Corpus Segmentation and Semantic Indexing:**
   - The acquired document corpus is segmented into smaller chunks to enhance precision in the retrieval process.
   - Each chunk of the document corpus is converted to a vector format by a sentence embedding model.
   - The vectors are utilized to represent the semantic meaning within the document corpus and to allow for similarity tests to be performed on the query and document content.
   - All the vectors are stored within a FAISS vector index to allow for semantic searches to be performed on the document corpus.
6. **User Query Processing:**
   - The query is submitted by the user through the web interface.
   - Preprocessing steps are carried out to clean and normalize the query.
   - The query analysis process begins.
7. **Information Source Selection:**
   - The knowledge source for the query will be selected based on the query type.
   - The faculty metadata dataset will be used to resolve faculty-related queries.
   - The location dataset will be used to resolve campus location-related queries.
   - The FAQ dataset will be used to resolve frequently asked questions.
8. **Semantic Retrieval for Complex Queries:**
   - When the query's answer is not immediately apparent from structured data or FAQs, the system uses semantic search over the document corpus to answer the query.
   - The query provided by the user is converted into a vector, which then points us to the most relevant text chunks stored within the FAISS index.
   - These retrieved text chunks provide us with the context to answer the query appropriately.

9. **Response Generation:**
   - The retrieved information will be structured into a coherent response.
   - A language model will be used to structure the information into a coherent response.
10. **Response Delivery to User Interface:**
   - The final response will be sent back to the web interface.
   - If the question relates to campus navigation, we will also display the user the relevant location on the interactive map.
   - The user will then be provided with the response generated along with relevant additional information.
11. **System Maintenance and Updates:**
   - The knowledge base is updated by adding new documents, updating FAQs, and updating faculty information.
   - Once new information has been added to the knowledge base, the corpus will be reprocessed, and the FAISS index will be updated to maintain the accuracy of the information retrieval process.
   - This process will allow the Campus Information Retrieval Assistant to combine structured information retrieval, semantic search, and contextual response generation.

VII. SIMULATION SETUP AND IMPLEMENTATION

The Campus Information Retrieval Assistant is an intelligent conversational interface intended to retrieve university information from various resources available within SR University. The intelligent conversational interface combines structured data, semantic search, and a SLM hosted locally to retrieve answers to queries related to university information. The intelligent conversational interface is built without any external dependencies on cloud-based APIs. This means that all operations, data storage, and model inference take place locally within the university environment.

The intelligent conversational interface is built using an implementation pipeline that transforms raw university data such as faculty data, university administration data, locations on campus, frequently asked questions, and university academic policy into a structured data set. The structured data is then combined with vector search and retrieval to ensure accurate and efficient query answering. The final product is a web-based interface linked to a server that accepts queries using various resolution layers.

1. EVOLUTION FRAMEWORK

The intelligent conversational interface is built on a modular framework that includes data collection, data processing, semantic indexing, data retrieval, model inference, and frontend deployment. The framework is built to output structured data such as a JSON data set, a vector index, or a model weight.

The system uses an eight-tier resolution hierarchy. First, deterministic techniques such as exact string matching and knowledge base lookup are used. If the query is not resolvable using these fast techniques, the system gradually moves to semantic search using vector embeddings and then to generative inference using the fine-tuned SLM.

1.1.SOFTWARE STRUCTURE

- Programming Language: Python 3.10
- Frameworks: FastAPI
- Libraries Used: NumPy, Requests, BeautifulSoup, PyMuPDF, Sentence Transformers, FAISS, RapidFuzz, Rank-BM25, Scikit-Learn
- Frontend Technologies: HTML, CSS, JavaScript
- Platform: VS Code, Jupyter Notebook
- Operating System: Windows 10/ Windows 11/ Linux

1.2.HARDWARE STRUCTURE

- GPU: Not required (CPU-based interface)
- Processor: Intel i5/ Ryzen 5 and above
- RAM: 8GB to16GB DDR4 RAM
- Network: Local Campus Network
- Storage: 4GB disk space is required

2. DATASET COLLECTION

- The system collects its required knowledge base through various official sources of the university.
- Faculty and university leadership information is collected from the SR University website. This information includes names, titles, departments, and profile image URLs for each faculty and university leadership personnel.
- Campus location information is collected manually and involves mapping the university campus locations such as academic blocks, libraries, canteens, hostels, and university administration offices onto the university campus image. The pixel coordinates for each location are recorded manually.
- FAQs are collected manually based on various sources such as student queries, inquiries made during orientation sessions, and official university announcements.
- University regulations and other academic and administrative information are collected from official university PDFs such as university regulations, exam regulations, hostel rules, and university curricula.
- All the collected information is stored in the form of structured data sets in the form of JSON and text data for further processing.

3. IMPLEMENTATION

3.1.WEB SCRAPING

- For the extraction of the information about the faculty members, the university's site is scraped, i.e., SR University's site is scraped, and the information about the staff members is obtained by the Python script.
- The information about the staff members is obtained by accessing the pages of the departments, and the information about the staff members is obtained by accessing the pages of the staff members individually.
- The information about the chancellor, the vice chancellor, the registrar, and the other staff members of the university is obtained by accessing the pages of the administration department.
- The information obtained from the webpage is normalized, i.e., the information obtained from the webpage is made uniform so that the information obtained from the webpage has the same fields.

Listing 1: Web Scraping Code

```
import requests

from bs4 import BeautifulSoup

url = "https://sru.edu.in/faculty/deptFaculties/cse"

response = requests.get(url)

html = response.text

soup = BeautifulSoup(html, "html.parser")

faculty_cards = soup.find_all("div", class_="card")

faculty_data = []

for card in faculty_cards:

    name = card.find("h3").text.strip()

    designation = card.find("p").text.strip()

    profile_link = card.find("a")["href"]

    image_link = card.find("img")["src"]

    faculty_data.append({
```

```
        "name": name,
        "designation": designation,
        "profile_link": profile_link,
        "image_link": image_link
    })
cleaned_entry = {
    "id": generate_slug(normalized_name),
    "name": normalized_name,
    "designation": clean_text(entry.get("designation")),
    "department": department,
    "school": title_case(clean_text(entry.get("school"))),
    "profile_link": clean_text(entry.get("profile_link")),
    "image_link": clean_text(entry.get("image_link")),
    "source": "SR University Official Website"
}
with open("faculty_metadata.json", "w", encoding="utf-8") as f:
    json.dump(cleaned, f, indent=2, ensure_ascii=False)
```

3.2.CAMPUS LOCATION DATA COLLECTION

- To drive the campus map navigation, we need to have accurate coordinate data on all the facilities on the map.
- By clicking on each facility on the map, we have recorded the pixel coordinate data.
- The various facilities such as academic block, library, hostels, auditorium, canteen, gym, parking, innovation centers, etc., have been collected in a data set named locations.json.
- Each entry in the data set contains the location name, the pixel coordinate data, and optional semantic data to help us identify the query such as “food,” “study,” “sports,” etc.

Code:

```
def on_click(event):
    cx = canvas.canvasx(event.x)
    cy = canvas.canvasy(event.y)
    ix = int(cx / scale)
    iy = int(cy / scale)
    name = simpledialog.askstring("Location Name", "Enter location name:")
    tag = simpledialog.askstring("Tag", "Enter tag (optional):")
    locations[name.title()] = {
        "x": ix,
        "y": iy,
        "tag": tag
    }
    with open("locations.json", "w") as f:
        json.dump(locations, f, indent=4)
    render_image()
```

3.3. FAQ AND KNOWLEDGE BASE

- We have developed a highly organized FAQ dataset that can provide fast and accurate answers to the most common student queries.
- The dataset contains questions and their accurate answers along with the required keywords for the topics discussed in the answers.
- Furthermore, the knowledge base section stores static university-related information such as the founding year of the university, its accreditation status, emails of the university authorities, and the university website.
- This section uses the exact matching of strings for fast response generation and does not rely on any machine learning model.

Code:

```
import json

# Load FAQ knowledge base

with open("sru_faqs.json", "r", encoding="utf-8") as f:

    faq_data = json.load(f)

def search_faq(query):

    query = query.lower()

    for item in faq_data:

        for keyword in item["keywords"]:

            if keyword.lower() in query:

                return item["response"]

    return None
```

3.4. PDF CORPUS CONSTRUCTION

- A significant number of university policies and academic guidelines are available in PDF format.
- Therefore, we have developed a system that can process these PDFs and construct the required corpus for the system.
- We have collected various PDFs such as academic regulations, exam guidelines, hostel rules and regulations, placement guidelines, and university curriculum from the official university website and other sources.
- The content of these PDFs has been extracted using the PyMuPDF library and cleaned using various techniques such as the removal of headers and footers and the removal of encoding and formatting issues.
- The cleaned content has been stored in a single file for the corpus. The corpus has been split into small chunks of text containing around 100 words for efficient querying and response generation while retaining the required section headers in the text chunks.

Code:

```
import fitz

import os

DATA_FOLDER = "data"

OUTPUT_FILE = "clean_corpus.txt"

with open(OUTPUT_FILE, "w", encoding="utf-8") as corpus:

    for file in os.listdir(DATA_FOLDER):

        if file.endswith(".pdf"):

            pdf_path = os.path.join(DATA_FOLDER, file)

            doc = fitz.open(pdf_path)

            for page in doc:

                corpus.write(page.get_text() + "\n")
```

3.5. VECTOR INDEX CONSTRUCTION

- To support semantic search, text data is converted into vector representations.
- We utilize the All Mini LM L6 v2 sentence transformer for dense vector generation for faculty records, FAQs, and chunks of corpus text data.
- The dense vectors encode semantic relationships between the user query and the stored data.
- The vectors are stored in the FAISS index for fast similarity search over thousands of vectors.
- We also provide support for keyword-based search using the BM25 algorithm.
- The semantic and keyword-based search systems run in parallel and combine the results using the reciprocal rank fusion algorithm for the most accurate answer.

Code:

```
import json

import faiss

from sentence_transformers import SentenceTransformer

import numpy as np

with open("faculty_metadata.json", "r", encoding="utf-8") as f:

    faculty_data = json.load(f)

model = SentenceTransformer("all-MiniLM-L6-v2")

texts = [

    f"{f['name']} {f['designation']} {f['department']}"  for f
in faculty_data ]

embeddings = model.encode(texts)

dimension = embeddings.shape[1]

index = faiss.IndexFlatL2(dimension)

index.add(np.array(embeddings))

faiss.write_index(index, "faculty_faiss.index")
```

3.6. SLM FINE-TUNING

- The generative part of the system employs a SLM, which is a variant of the GPT-2 language model.
- This language model is fine-tuned on question-answer pairs from clean documents.
- In the process of fine-tuning, section titles and key facts are used to generate questions, while the corresponding documents serve as the answer to the questions.
- The language model is stored locally after the process and is loaded by the server at system startup.
- The language model generates an answer to the query only when other deterministic and retrieval-based techniques fail to confidently answer the query.
- This approach keeps the generative answer within the scope of official university documents and prevents hallucinations in the answer.

Code:

```
from transformers import AutoModelForCausalLM, AutoTokenizer, Trainer, TrainingArguments

from datasets import Dataset

import json

BASE_MODEL = "Qwen/Qwen2.5-0.5B-Instruct"

OUTPUT_DIR = "models/club-slm"

# Load QA training dataset

with open("train_qa.jsonl", "r", encoding="utf-8") as f:

    data = [json.loads(line) for line in f]

dataset = Dataset.from_list(data)

# Load tokenizer and model

tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)

model = AutoModelForCausalLM.from_pretrained(BASE_MODEL)
```

```
# Tokenization function

def tokenize(example):

    text = f"Question: {example['question']}\nAnswer: {example['answer']}"

    return tokenizer(text, truncation=True, padding="max_length", max_length=256)

tokenized_dataset = dataset.map(tokenize)

# Training configuration

training_args = TrainingArguments(

    output_dir=OUTPUT_DIR,

    num_train_epochs=5,

    per_device_train_batch_size=2,

    learning_rate=3e-5,

    logging_steps=10,

    save_strategy="epoch"

)

trainer = Trainer(

    model=model,

    args=training_args,

    train_dataset=tokenized_dataset

)

trainer.train()

# Save final fine-tuned model

trainer.save_model(OUTPUT_DIR)
```

3.7. SYSTEM INTEGRATION AND DEPLOYMENT

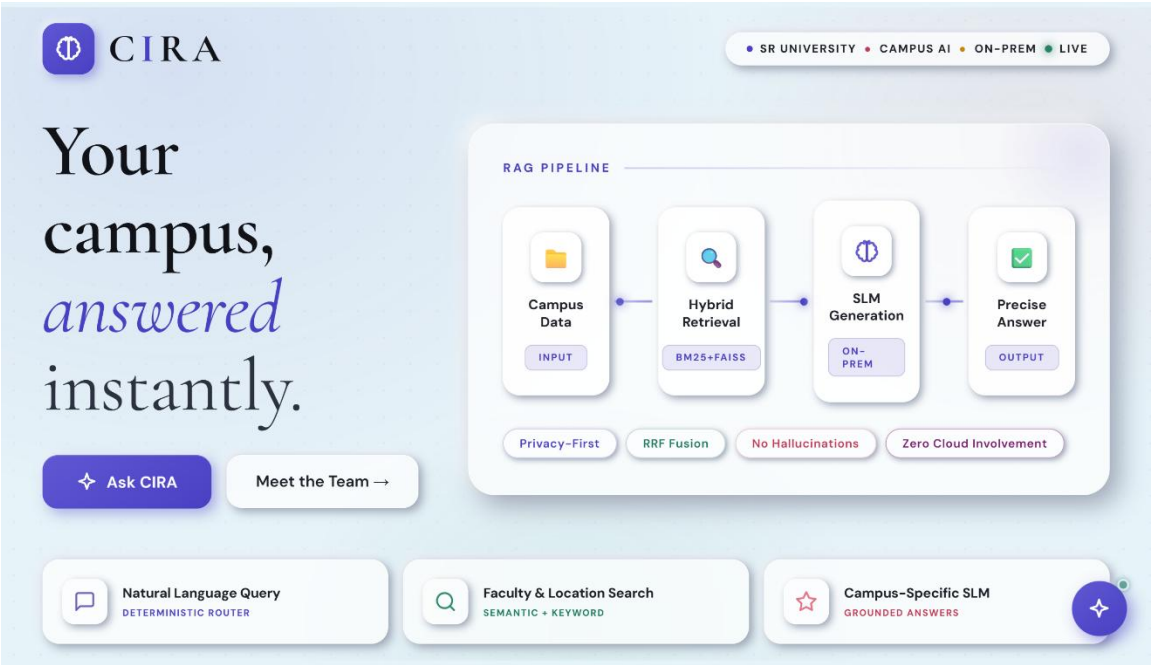
- The system's components integrate into a single system on a backend server, which is built using FastAPI. The backend has a chat endpoint to process the user's questions.
- The chat endpoint processes the user's questions through a resolution process.
- The system processes the questions to identify if they are related to faculty information, location on the campus, FAQs, knowledge base, or documents.
- For the frontend, there is an HTML file with integrated CSS and JavaScript. The frontend has features such as a chat panel, faculty information on cards, and a map of the campus.
- The system runs entirely locally on a machine and can be initialized with a single command once all dependencies are installed.
- Once it is initialized on a network, students and staff can access it through any web browser.

Code:

```
from fastapi import FastAPI

from pydantic import BaseModel

from query_engine import process_query

app = FastAPI()

class Query(BaseModel):

    question: str

@app.post("/ask")

def ask_question(q: Query):

    response = process_query(q.question)

    return {"answer": response}

if __name__ == "__main__":

    import uvicorn

    uvicorn.run(app, host="127.0.0.1", port=8000)
```

4. USER INTERFACE

- The frontend of the system has been built in such a manner that the user interface appears as natural as possible.
- Users can ask the system any query pertaining to university information in the form of a chat panel in the user interface.
- The system displays the response in the form of text or cards based on the query asked by the user.
- If the user asks any query pertaining to faculty information, the system displays the faculty information in the form of cards containing the image of the faculty member, the name of the faculty member, and the official faculty webpage link.
- If the user asks any query pertaining to the location of any facility within the university campus, the system displays the location in the form of an interactive campus map that marks the location of the facility and other locations nearby.
- The user interface has been built using only HTML, CSS, and JavaScript and can run directly in the browser without the need for any further installation.



Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"><meta name="viewport"
content="width=device-width,initial-scale=1.0">

<title>CIRA — SR University</title>

<link
href="https://fonts.googleapis.com/css2?family=Cormorant+Garam
ond:ital,wght@0,600;0,700;1,400&family=DM+Sans:wght@400;600;70
0&display=swap" rel="stylesheet">

<style>

:root { --bg:#e6f3f8; --accent:#4a40c4; --ink:#111318; --
glass:rgba(255,255,255,0.72); }

* { box-sizing: border-box; margin: 0; padding: 0; font-
family: "DM Sans", sans-serif; }

body { background: var(--bg); color: var(--ink); min-height:
100vh; overflow-x: hidden; }

.page { max-width: 1250px; margin: 0 auto; padding: 0 44px; }

nav { display: flex; justify-content: space-between; align-
items: center; padding: 22px 0; }

.logo { display: flex; align-items: center; gap: 15px; text-
decoration: none; }

.mark { width: 45px; height: 45px; border-radius: 12px;
background: linear-gradient(145deg, #6058d4, #4a40c4); color:
white; display: flex; align-items: center; justify-content:
center; font-weight: bold; font-size: 24px; box-shadow: 4px
4px 10px rgba(74,64,196,0.38); }

.word { font-family: "Cormorant Garamond", serif; font-size:
2.6rem; font-weight: 700; } .word span { color: var(--accent);
}
```

```
.badge { display: flex; align-items: center; gap: 8px;
background: var(--glass); padding: 9px 22px; border-radius:
100px; font-size: 0.75rem; font-weight: 700; text-transform:
uppercase; border: 1px solid rgba(255,255,255,0.7); box-
shadow: 4px 4px 10px #9aa3b4, -4px -4px 10px #eef0f6; }

.dot { width: 8px; height: 8px; border-radius: 50%;
background: #1d7a5e; box-shadow: 0 0 10px #1d7a5e; animation:
p 2s infinite; }

@keyframes p { 50% { opacity: 0.5; transform: scale(0.8); } }

.hero { padding: 80px 0; max-width: 600px; }

h1 { font-family: "Cormorant Garamond", serif; font-size:
clamp(4rem, 6.5vw, 5.9rem); line-height: 1.05; margin-bottom:
32px; }

h1 em { color: var(--accent); font-style: italic; font-weight:
400; } h1 .soft { color: #4a5068; font-weight: 400; }

.cta { display: flex; gap: 16px; }

.btn { padding: 17px 36px; border: none; border-radius: 16px;
background: linear-gradient(145deg, #6058d4, #4a40c4); color:
white; font-weight: 600; font-size: 1.05rem; cursor: pointer;
box-shadow: 5px 5px 14px rgba(74,64,196,0.4); }

.btn-sec { padding: 16px 30px; border-radius: 16px;
background: var(--glass); border: 1px solid
rgba(255,255,255,0.8); font-weight: 600; color: #1e2128;
cursor: pointer; box-shadow: 4px 4px 10px #9aa3b4, -4px -4px
10px #eef0f6; }

.fab { position: fixed; bottom: 26px; right: 26px; width:
58px; height: 58px; border-radius: 50%; background: linear-
gradient(145deg, #6058d4, #4a40c4); color: white; border:
none; cursor: pointer; box-shadow: 6px 6px 16px
rgba(74,64,196,0.48); display: flex; align-items: center;
justify-content: center; transition: transform 0.2s; }
```

```
.fab svg { width: 24px; height: 24px; fill: none; stroke:
#fff; stroke-width: 2.2; stroke-linecap: round; stroke-
linejoin: round; }

.fab:hover { transform: scale(1.08); }

</style> </head>

<body>

<div class="page">

  <nav>

    <div class="logo"><div class="mark">C</div><div
class="word">C<span>I</span>RA</div></div>

    <div class="badge">SR University <span class="dot"></span>
Live</div>  </nav>

  <section class="hero">

    <h1>Your campus,<br><em>answered</em><br><span
class="soft">instantly.</span></h1>

    <div class="cta">

      <button class="btn">Ask CIRA</button>

      <button class="btn-sec">Meet the Team →</button>

    </div>

  </section>

</div>

<button class="fab">

  <svg viewBox="0 0 24 24"><path d="m12 3-1.912 5.813a2 2 0 0
1-1.275 1.275L3 12l5.813 1.912a2 2 0 0 1 1.275 1.275L12
21l1.912-5.813a2 2 0 0 1 1.275-1.275L21 12l-5.813-1.912a2 2 0
0 1-1.275-1.275L12 3Z"/></svg>

</button>

</body>

</html>
```

VIII. RESULT COMPARISON AND ANALYSIS

8.1. Evaluation of FAQ-Based Retrieval

Queries for information that is commonly sought by students, such as information about the university, academic programs, university rankings, and admission requirements, are handled by the FAQ knowledge base. However, the keyword-based retrieval system proved to be very efficient in handling such queries since results are obtained instantaneously with very high accuracy.

8.2. Performance of Faculty Information Retrieval

Queries related to faculty information are evaluated using the semantic search feature, which is implemented using FAISS. Since vector representation is used for storing faculty information, even if a query is not similar in terms of words, using vector representation, relevant faculty information is obtained. For example, if a query is "Who is the HOD of CSE?" or "Head of Computer Science department," relevant faculty information is obtained.

8.3. Campus Location Retrieval in a Nutshell

We validated our campus location retrieval model by utilizing a manually annotated data set that contained campus locations. For instance, locations like the Central Library, Auditorium, or Canteen came with associated pixel coordinates and tags. When users asked for a particular location, our model matched the tags and located the correct coordinates. Our frontend then placed a marker on the campus map, allowing users to easily locate the location.

8.4. Evaluating the SLM

We evaluated our fine-tuned version of the SLM by utilizing queries that came from official documents and reliable sources. Our model proved to be effective in providing concise and context-aware answers to users even if the answer was not included in the FAQ. Our model scored a Perplexity of 1.3984, indicating that our model has excellent predictive capabilities and has learned effectively from the training data.

8.5. Retrieval Techniques

Our model, which combined FAQ, semantic search, and language model-based retrieval, outperformed other techniques. FAQ-based retrieval proved to be effective in providing answers to predefined FAQs, semantic search proved to be effective in retrieving accurate faculty information, and language model-based search proved to be effective in retrieving answers to descriptive-based queries.

8.6. Response Time Dynamics

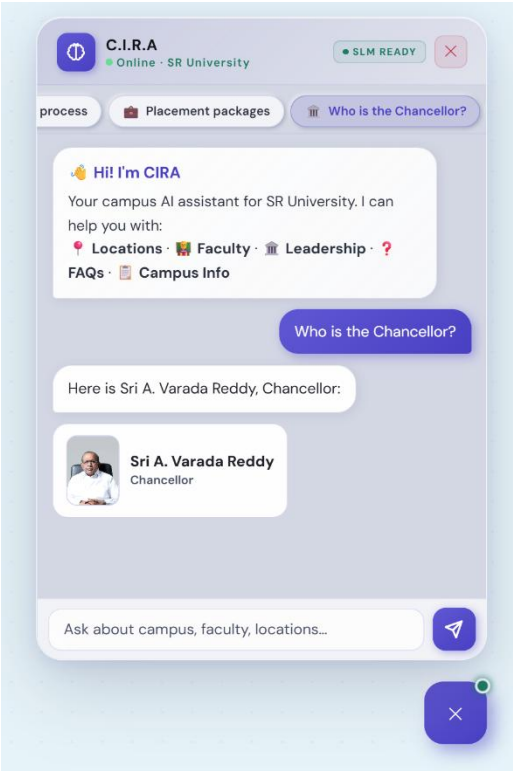
Our model proved to be efficient in providing responses to users. For instance, our model proved to be instantaneous in retrieving answers to keyword-based queries, our semantic search model proved to be efficient in retrieving answers to semantic search queries, and our language model proved to be efficient in providing responses to users within a short interval of time.

8.7. Accuracy and Relevance

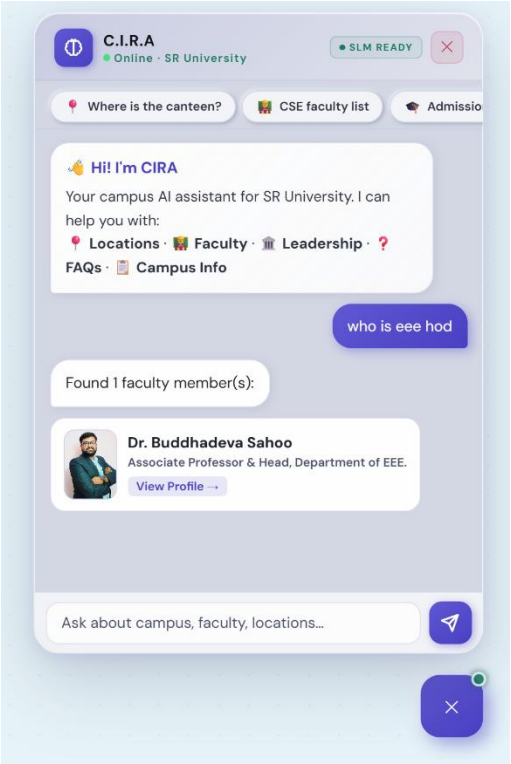
Our model proved to be accurate and relevant in providing answers to users. Our model proved to be relevant in providing answers to users because of the integration of structured data and semantic search. Our model proved to be accurate in providing answers to users because of the integration of structured data and semantic search.

8.8. Overall System Effectiveness

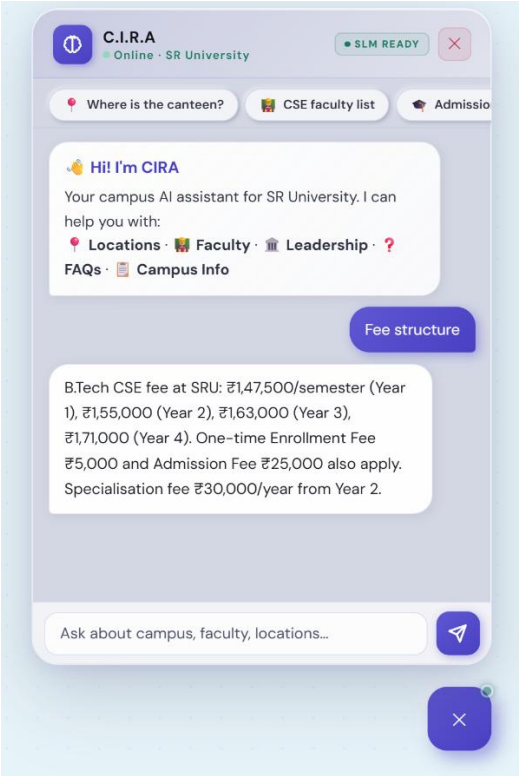
Our model proved to be effective in providing answers to users. Our model proved to be effective in providing answers to users because of the integration of FAQ, semantic search, and language model-based retrieval. Our model proved to be effective in providing answers to users because of the integration of FAQ, semantic search, and language model-based retrieval.



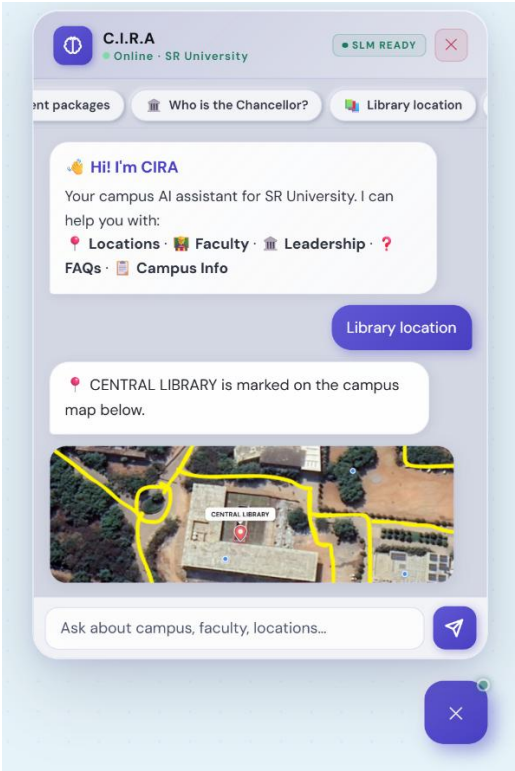
Hierarchy Search



Faculty Search



Information Search



Place Search

IX. LEARNING OUTCOME

1. Overview of Intelligent Information Retrieval Systems

sThis project provided a hands-on learning experience in developing and implementing an intelligent information retrieval system that meets the requirements of campus-based information queries.

2. Data Collection and Knowledge Base Development

The Data was collected from Official Sources of SRU like University Mail, Social Groups, etc., The Knowledge base developed With FAISS Indexing.

3. Web scraping techniques

These were used to fetch data automatically, and the students learned to use these techniques to extract information related to faculty and institutions.

4. Foundations of Natural Language Processing

The project introduced the students to various concepts in NLP, including converting text to vector representations and semantic similarity to fetch relevant information from large data sets.

5. Vector Search with FAISS

The students learned to develop vector representations and FAISS to fetch relevant information, even if the query is not similar to the data.

6. Training and Fine-Tuning a SLM

The students learned to develop a SLM and fine-tune the model to fetch relevant information based on various documents.

7. Integration of Multiple AI Components

A major takeaway was to understand how disparate components, such as FAQ retrieval, semantic search, and language model inference, can be combined to form a cohesive system architecture.

8. Backend Development and API Deployment

Students were able to learn how to develop backend services for handling user queries, as well as integrating the system through APIs for fast response.

9. User Interface and System Interaction Design

Designing the web interface and campus map helped students learn more about user interaction design for an intelligent system.

X. CONCLUSION

The Campus Information Retrieval Assistant (CIRA) was born as a smart system with the capability to provide answers to campus-related questions by combining structured data, semantic search techniques, and a well-tuned Small Language Model.

This project demonstrates the practical application of modern AI techniques in the real world within the context of an educational setting. By combining keyword-based search techniques with vector similarity search using the FAISS library and natural language generation using the fine-tuned model, the system can effectively handle the wide range of questions that can be asked by the end-user.

A significant achievement of this project is the campus navigation system that can pinpoint the exact location of the facility on the map. This visual representation of the information can help the end-user find the exact facility they are looking for, such as the academic blocks, hostels, or administrative offices.

The intelligent assistant is particularly useful as it can display campus location on a map. The hybrid mechanism is particularly useful as it can be extended with new features, new datasets, etc., in the future. The example is particularly useful as it illustrates how intelligent assistants can be built using various techniques

In conclusion, this project presents a practical and effective solution to the development of automated information systems within the context of the university setting. It can be improved by adding more datasets or features in the future by using the modular nature of the system. It demonstrates the practical application of the development of intelligent assistants that can effectively provide the end-user with fast access to the information they are looking for.

CHALLENGES FACED:

1. **Data Collection and Consistency:**

   Accurate and well-organized data collection from various university sources was found to be challenging. University data was scattered across many different web pages with different formats.

2. **Data Collection and Consistency:**

   Accurate and well-organized data collection from various university sources was found to be challenging. University data was scattered across many different web pages with different formats.

3. **Web Scraping Limitations:**

   In obtaining faculty information from the website, web scraping was involved. The problem was that web pages sometimes change their presentation, and this could cause problems during web scraping.

4. **Dataset Preparation and Cleaning:**

   Preparing and cleaning the obtained web pages and PDFs involved pre-processing. The pre-processing involved removing unwanted text and formatting inconsistencies. The pre-processing also involved removing duplicates from the obtained text.

5. **Handling Large Document Corpus**

   Transforming many PDFs from the university into one text body was challenging. This was due to the need for text chunking and extraction so that the text could be used effectively.

6. **Model Training Constraints:**

   Training the small language model had constraints. The constraints were mainly related to hardware limitations, especially memory constraints. This was to enable smooth execution of the small language model.

7. **Integration of Multiple System Components:**

   Integration of all the system components, including FAQs, semantic search, small language model inference, and campus map navigation, was involved. This was challenging and required careful design and debugging.

XI. REFERENCES

[1] A. Nguyen-Duc, C. V. Manh, B. A. Tran, V. P. Ngo, L. L. Chi, and A. Q. Nguyen, "An Empirical Study of Multi-Agent RAG for Real-World University Admissions Counseling," arXiv, 2025.

DOI: https://doi.org/10.48550/ARXIV.2507.11272

Summary: This work studies the use of multi-agent Retrieval-Augmented Generation systems to provide accurate and context-aware guidance for university admission counselling.

[2] J.-K. Oh and S.-K. Kim, "Design of a RAG-Based Q&A System for University Academic Administration Assistants," Journal of Korea Society of Computer and Information (JKSci), vol. 30, no. 7, pp. 79–86, 2025.

DOI: https://doi.org/10.9708/jksci.2025.30.07.079

Summary: The paper proposes a RAG-based question answering system designed to assist academic administration by retrieving institutional information efficiently.

[3] S. Xu, Z. Yan, C. Dai, and F. Wu, "MEGA-RAG: A Retrieval-Augmented Generation Framework with Multi-Evidence Guided Answer Refinement for Mitigating Hallucinations of LLMs in Public Health," Frontiers in Public Health, vol. 13, p. 1635381, 2025.

DOI: https://doi.org/10.3389/fpubh.2025.1635381

Summary: This research introduces a multi-evidence guided RAG framework to reduce hallucination problems in large language model responses.

[4] K. Anjali, K. Vipunsai, K. Ruchitha, M. Bhavani, and Ch. China Subba Reddy, "Privacy Centric Offline Chatbot Using Large Language Models," International Transactions on Electrical Engineering and Computer Science, vol. 4, no. 2, pp. 119–126, 2025.

DOI: https://doi.org/10.62760/iteecs.4.2.2025.134

Summary: This paper presents an offline chatbot system using language models with a focus on privacy protection and local data processing.

[5] L. S. Hartono, E. I. Setiawan, and V. Singh, “Retrieval Augmented Generation-Based Chatbot for Prospective and Current University Students,” International Journal of Engineering Science and Information Technology, vol. 5, no. 3, pp. 268–277, 2025.

DOI: https://doi.org/10.52088/ijesty.v5i3.951

Summary: The study develops a university chatbot using RAG architecture to assist students in obtaining institutional information and academic guidance.

[6] J. Swacha and M. Gracel, “Retrieval-Augmented Generation (RAG) Chatbots for Education: A Survey of Applications,” Applied Sciences, vol. 15, no. 8, p. 4234, 2025.

DOI: https://doi.org/10.3390/app15084234

Summary: This survey reviews various applications of RAG-based chatbots in educational environments and highlights their potential benefits.

[7] S. Dong, Z. Cai, C. Wu, and C. Ren, “Sharing Quantum Steering via Standard Projective Measurements,” arXiv, 2024.

DOI: https://doi.org/10.48550/ARXIV.2405.08588

Summary: The study explores quantum steering techniques and their implementation using standard projective measurement methods.

[8] S. Subramanian, V. Elango, and M. Gungor, “SLMs (SLMs) Can Still Pack a Punch: A Survey,” arXiv, 2025.

DOI: https://doi.org/10.48550/ARXIV.2501.05465

Summary: This survey highlights the effectiveness of SLMs and their ability to perform efficient domain-specific tasks.

[9] I. Lamaakal, Y. Maleh, K. El Makkaoui, I. Ouahbi, P. Pławiak, O. Alfarraj, M. Almousa, and A. A. Abd El-Latif, “Tiny Language Models for Automation and Control: Overview, Potential Applications, and Future Research Directions,” Sensors, vol. 25, no. 5, p. 1318, 2025.

DOI: https://doi.org/10.3390/s25051318

Summary: The paper discusses the role of tiny language models in automation systems and outlines potential future research directions.

[10] P. Zhang, G. Zeng, T. Wang, and W. Lu, “TinyLlama: An Open-Source SLM,” arXiv, 2024.

DOI: https://doi.org/10.48550/ARXIV.2401.02385

Summary: This work introduces TinyLlama, an efficient open-source SLM designed for lightweight AI applications.

[11] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020.

DOI: https://doi.org/10.48550/arXiv.2005.11401

Summary: This foundational research introduces the Retrieval-Augmented Generation architecture, combining neural language models with external knowledge retrieval.

PROJECT ARTIFACTS AND SUPPORTING LINKS

Source Code Repository

https://github.com/jayanthbottu/Campus-Information-Retrieval-Assistant

Demo:

https://jayanthbottu.github.io/CIRA/

Datasets:

SRU Official Documents, Students Handbook, SRU Newsletter,

sru.edu.in for Faculty and FAQ Information

Technical Blog Article:

https://medium.com/@jayanthbottu/c-i-r-a-111c2f6de0cb

Social Media & Outreach:

https://www.linkedin.com/feed/update/urn:li:activity:7440019180956844032/

Walkthrough Video:

https://youtu.be/Mw3KNy5wPy8